

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behaviour. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
 - (b) Design a responsive navigation bar using appropriate CSS techniques.
 - (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
 - (d) Evaluate the usability and suggest improvements for better user experience.
-

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
 - (b) Suggest appropriate **CSS animation or transition** for product interaction.
 - (c) Design a hover effect for a product card.
 - (d) Evaluate performance and user experience considerations.
-

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).

- (b) Compare **Flexbox and Grid** based on layout needs.
 - (c) Choose the most suitable layout technique and justify your choice.
 - (d) Evaluate the scalability and responsiveness of the chosen approach.
-

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
 - (b) Design a mobile-first layout using CSS.
 - (c) Demonstrate how the layout scales for larger screens using media queries.
 - (d) Evaluate accessibility and readability of the design.
-

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

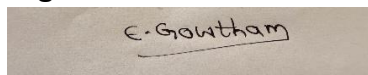
Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Code of Conduct:

I Enamala Gowtham /192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in black ink that reads "E. Gowtham".

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design

(a) Key components required in an e-commerce navigation bar

A responsive e-commerce navigation bar should contain:

1. **Logo/Brand name** – identifies the website.
2. **Home** – provides access to the main page.
3. **Product/Categories** – helps users browse products.
4. **Search bar** – allows users to quickly find products.
5. **Cart icon** – displays selected products.
6. **Login/Account** – provides access to user account.
7. **Hamburger menu** – used on smaller screens to display navigation links.

(b) Responsive navigation bar using CSS

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
* {  
    box-sizing: border-box;  
}
```

```
body {  
    margin: 0;  
    font-family: Arial, sans-serif;  
}
```

```
.navbar {  
    display: flex;  
    align-items: center;  
    justify-content: space-between;  
    padding: 15px 30px;  
    background: #222;  
}
```

```
.logo {  
    color: white;  
    font-size: 22px;  
    font-weight: bold;  
}
```

```
.nav-links {  
    display: flex;  
    gap: 25px;  
}
```

```
.nav-links a {  
  color: white;  
  text-decoration: none;  
}
```

```
.search {  
  padding: 8px;  
  border-radius: 4px;  
  border: none;  
}
```

```
.menu-btn {  
  display: none;  
  color: white;  
  font-size: 25px;  
  cursor: pointer;  
}
```

```
@media (max-width: 768px) {  
  .nav-links {  
    display: none;  
    flex-direction: column;  
    width: 100%;  
  }
```

```
  .nav-links.active {  
    display: flex;  
  }
```

```
  .navbar {  
    flex-wrap: wrap;  
  }
```

```
  .menu-btn {  
    display: block;  
  }
```

```
  .search {  
    width: 100%;  
    margin-top: 10px;  
  }  
}  
</style>
```

```

</head>

<body>
<nav class="navbar">
  <div class="logo">ShopEasy</div>
  <span class="menu-btn" onclick="toggleMenu()">≡</span>
  <div class="nav-links" id="navLinks">
    <a href="#">Home</a>
    <a href="#">Products</a>
    <a href="#">Categories</a>
    <a href="#">Cart</a>
    <a href="#">Account</a>
  </div>
  <input class="search" type="text" placeholder="Search products">
</nav>
<script>
function toggleMenu() {
  document.getElementById("navLinks").classList.toggle("active");
}
</script>
</body>
</html>

```

(c) How the navigation adapts to smaller screens

A media query is used:

@media (max-width: 768px)

On mobile devices:

- Normal navigation links are hidden.
- A hamburger menu (≡) is displayed.
- Clicking the hamburger displays the navigation links.
- The search box uses the available screen width.

Thus, the navigation changes from a horizontal desktop layout to a compact mobile layout.

(d) Usability and improvements

The navigation is easy to use because important options such as Products, Search, Cart, and Account are easily accessible.

Improvements include:

- Use clear icons for cart and account.
- Provide visible focus indicators for keyboard users.
- Use readable font sizes.
- Make buttons and links large enough for touch interaction.
- Keep navigation consistent across all pages.

QUESTION 2: Product Hover Effect

(a) Suitable CSS properties for hover effects

Suitable CSS properties include:

- transform
- transition
- box-shadow
- opacity
- background-color
- border
- filter
-

(b) Suitable CSS animation or transition

A smooth transition can be created using:

transition: transform 0.3s ease, box-shadow 0.3s ease;

The product card can slightly move upward and display a shadow when the mouse pointer moves over it.

(c) Design a hover effect for a product card

```
<!DOCTYPE html>
<html>
<head>
<style>
.product-card {
  width: 250px;
  padding: 15px;
  margin: 30px;
  text-align: center;
  border: 1px solid #ddd;
  border-radius: 10px;
  transition: transform 0.3s ease,
             box-shadow 0.3s ease;
}

.product-card img {
  width: 100%;
  border-radius: 8px;
}

.product-card:hover {
  transform: translateY(-8px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);
}
```

```

}

.product-card button {
  padding: 10px 20px;
  border: none;
  cursor: pointer;
}
</style>
</head>

<body>

<div class="product-card">
  
  <h3>Smart Watch</h3>
  <p>₹1999</p>
  <button>Add to Cart</button>
</div>

</body>
</html>

```

When the mouse moves over the card, `.product-card: hover` is activated. The card moves upward and gets a shadow.

(d) Performance and user experience

For good performance:

- Use simple CSS transitions.
- Avoid complex animations.
- Keep transition duration short.
- Use optimized images.
- Avoid unnecessary effects.

For good user experience, the hover effect should be smooth and noticeable but not distracting.

The design should also consider touch devices because traditional hover may not work on them.

QUESTION 3: Layout Selection – Flexbox vs Grid

(a) Dashboard layout requirements

A dashboard generally requires:

- Header at the top.
- Sidebar for navigation.
- Main content area.
- Multiple rows and columns.
- Cards for displaying information.
- Proper alignment and spacing.
- Responsive behavior for different screen sizes.

Example:

```
-----  
Header  
-----  
Sidebar | Main Content  
        | Card  Card  
        | Card  Card  
-----
```

(b) Compare Flexbox and Grid

Feature	Flexbox	CSS Grid
Layout	One-dimensional	Two-dimensional
Rows	Limited	Excellent
Columns	Limited	Excellent
Alignment	Very good	Very good
Dashboard	Suitable	Highly suitable
Complex layouts	Less suitable	More suitable
Responsive design	Good	Excellent

Flexbox is mainly used for arranging elements in a row or column.

CSS Grid is used for arranging elements in rows and columns simultaneously.

(c) Most suitable layout technique

CSS Grid is the most suitable technique for a dashboard because dashboards normally contain two-dimensional rows and columns.

Example:

```
.dashboard {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 70px 1fr;  
  min-height: 100vh;  
}
```

```

}

.header {
  grid-column: 1 / 3;
}

.sidebar {
  grid-column: 1;
}

.main {
  grid-column: 2;
}

```

(d) Scalability and responsiveness

CSS Grid is scalable because new rows and columns can be added easily.

For smaller screens:

```

@media (max-width: 768px) {
  .dashboard {
    grid-template-columns: 1fr;
    grid-template-rows: auto;
  }

  .header,
  .sidebar,
  .main {
    grid-column: 1;
  }
}

```

Therefore, the dashboard can adapt to desktop, tablet, and mobile screens.

QUESTION 4: Mobile-First Blog Layout

(a) Key sections of a blog layout

A blog page can contain:

1. Header
2. Navigation
3. Main article
4. Article title
5. Article content
6. Images
7. Sidebar
8. Footer

(b) Design a mobile-first layout using CSS

Mobile-first design means that the basic CSS is written for small screens first.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
* {  
  box-sizing: border-box;  
}
```

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
}
```

```
header {  
  padding: 20px;  
  text-align: center;  
  background: #222;  
  color: white;  
}
```

```
nav {  
  padding: 10px;  
  text-align: center;  
}
```

```
nav a {  
  display: block;  
  padding: 8px;
```

```
    text-decoration: none;
}
```

```
.container {
    width: 100%;
    padding: 15px;
}
```

```
article {
    width: 100%;
}
```

```
article img {
    width: 100%;
    height: auto;
}
```

```
aside {
    width: 100%;
    margin-top: 20px;
    padding: 15px;
}
```

```
footer {
    padding: 15px;
    text-align: center;
    background: #222;
    color: white;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
    <h1>My Tech Blog</h1>
```

```
</header>
```

```
<nav>
```

```
    <a href="#">Home</a>
```

```
    <a href="#">Technology</a>
```

```
    <a href="#">About</a>
```

```
</nav>
```

```
<div class="container">
```

```

<article>
  <h2>Introduction to Web Development</h2>
  <p>
    Web development involves creating and maintaining
    websites using HTML, CSS and JavaScript.
  </p>

  
</article>

<aside>
  <h3>Recent Posts</h3>
  <p>HTML Basics</p>
  <p>CSS Layout</p>
  <p>JavaScript Events</p>
</aside>

</div>

<footer>
  Copyright 2026
</footer>

</body>
</html>

```

(c) Scaling for larger screens using media queries

```

@media (min-width: 768px) {
  .container {
    display: grid;
    grid-template-columns: 2fr 1fr;
    gap: 30px;
    max-width: 1100px;
    margin: auto;
  }

  nav a {
    display: inline-block;
    margin: 0 10px;
  }
}

```

On mobile:

Article

Sidebar

On larger screens:

Article Sidebar

Article Sidebar

Thus, the design follows a mobile-first approach and scales using media queries.

(d) Accessibility and readability

The design should:

- Use semantic HTML such as <header>, <nav>, <article>, <aside>, and <footer>.
- Use alt text for images.
- Use readable font sizes.
- Maintain sufficient contrast.
- Provide clear headings.
- Use proper spacing.
- Make links accessible using the keyboard.
- Avoid very small text.

These practices improve accessibility and readability.

QUESTION 5: JavaScript Event Handling for Dynamic Menu

(a) Appropriate JavaScript events

Useful events for a dynamic menu include:

- click – opens or closes the menu.
- mouseover – responds when the pointer enters an element.
- mouseout – responds when the pointer leaves.
- keydown – supports keyboard interaction.

For a mobile hamburger menu, the **click event** is the most appropriate.

(b) Event handling logic

The logic is:

User clicks hamburger



JavaScript receives click event



Toggle menu class



Menu becomes visible/hidden

(c) Implement interaction for user actions

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.menu {  
    display: none;  
    background: #222;  
}
```

```
.menu.active {  
    display: block;  
}
```

```
.menu a {  
    display: block;  
    padding: 12px;  
    color: white;  
    text-decoration: none;  
}
```

```
#menuButton {  
    padding: 10px;  
    cursor: pointer;
```

```

}
</style>
</head>

<body>

<button id="menuButton">≡ Menu</button>

<div class="menu" id="menu">
  <a href="#">Home</a>
  <a href="#">Products</a>
  <a href="#">Services</a>
  <a href="#">Contact</a>
</div>

<script>
const menuButton = document.getElementById("menuButton");
const menu = document.getElementById("menu");

menuButton.addEventListener("click", function() {
  menu.classList.toggle("active");
});
</script>

</body>
</html>

```

The important statement is:

```
menu.classList.toggle("active");
```

It adds the active class when the menu is closed and removes it when the menu is open.

(d) Performance and improvements

For better performance:

- Use `addEventListener()`.
- Avoid unnecessary event handlers.
- Use CSS for visual effects instead of JavaScript.
- Keep JavaScript code simple.
- Use event delegation when handling many menu items.

For better accessibility:

- Use a real `<button>` for the menu.
- Support keyboard interaction.
- Provide meaningful labels.
- Maintain visible focus indicators.